CARAT: Class-Aware Robust Aggregation for Model-Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers aggregate client updates without seeing client data, which leaves training exposed to model-poisoning attacks. Most robust aggregators rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction. These signals weaken when benign clients are heterogeneous and adaptive attackers stay close to the benign update manifold. We propose **CARAT**, a class-aware robust aggregation rule built on *hidden task certificates*. Given a small labeled server-side reference set, CARAT samples multiple hidden, class-balanced micro-tasks and scores each client update by the loss reduction it achieves across them. CARAT combines these task-conditioned scores with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization. The result is an aggregator that favors updates that remain useful across diverse semantic slices while limiting the influence of structurally extreme clients. We frame CARAT specifically as a defense against untargeted model poisoning under heterogeneous client data and use a matched CIFAR-100/ResNet18 non-i.i.d. evaluation protocol with 20% malicious clients, public robust-aggregation baselines, matched trusted-data budgets, and class-sensitive metrics.

1 Introduction

Federated learning (FL) lets many clients train a shared model without centralizing raw data [1–3]. This decentralization is attractive in privacy- and governance-sensitive settings, but it leaves the server in an asymmetric position: the global model must be updated from client messages that the server cannot directly verify. The aggregation rule therefore becomes a security-critical part of the FL system. Malicious clients can slow convergence or distort the decision boundary through model-poisoning updates that remain difficult to distinguish from benign client drift [4–7].

The central difficulty is that benign heterogeneity and malicious behavior can look similar from the server's perspective. Under non-i.i.d. partitions, benign clients can differ substantially in update direction, norm, sparsity, and class emphasis [8–11]. An unusual update may therefore reflect either a harmful client or a legitimately different local distribution. This ambiguity weakens defenses based on coarse notions of centrality. Pairwise-distance methods can mistake benign dispersion for adversarial deviation, coordinate-wise robust statistics discard cross-coordinate structure, and trusted-anchor approaches compress all auxiliary supervision into a single global reference direction [12–19]. Such signals are often too coarse to expose attacks whose effect is concentrated on a subset of classes or semantic regions.

CARAT addresses this problem by shifting validation from geometric similarity to task-conditioned utility. The server maintains a small labeled reference set and repeatedly instantiates hidden, class-balanced micro-tasks from it. Each client update is scored by the loss reduction it induces across

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

these tasks, producing a certificate matrix that measures whether the update is broadly useful across semantic slices of the problem instead of merely close to the geometric center of the client population. CARAT then combines this signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization.

This design targets FL deployments in which a modest server-side audit set is realistic, such as cross-silo or institutionally managed settings. It is not a fully data-free defense. Within that setting, CARAT aims to preserve benign diversity while reducing the influence of updates that are semantically harmful or structurally extreme. Our main contributions are as follows.

- We propose CARAT, a class-aware robust aggregation rule that validates client updates through hidden-task certificates constructed from server-side labeled reference data, replacing one-shot geometric trust signals with multi-task utility checks.

- We formulate aggregation as a capped-simplex optimization problem that combines worst-task utility across hidden tasks, coordinate-rank anomaly suppression, and temporal regularization, yielding a robust weighting mechanism compatible with heterogeneous FL.

- We define a matched CIFAR-100/ResNet18 non-i.i.d. evaluation against public robust-aggregation baselines across four model-poisoning attacks under $\alpha \in \{1, 0.5\}$, with trusted-data budgets and seed counts reported explicitly.

2 Related Work

Model-poisoning attacks. Modern FL attacks are designed to remain effective under server-side defenses. Early work already showed that even constrained adversaries can poison the global model while remaining relatively stealthy [4–6]. In particular, attacks such as ALIE, MinMax, and MinSum craft malicious updates that evade simple geometric or statistical filters while still harming convergence or accuracy [6, 7]. Surveys make the broader point explicit: once the server cannot inspect raw client data, model poisoning becomes a systems concern rather than an edge case [3, 20].

Robust aggregation under heterogeneity. Classical Byzantine-robust aggregation rules infer benignness from centrality. Krum and related geometry-based methods favor updates that remain close to the client population under pairwise distances [12, 13], while coordinate-wise median and trimmed mean suppress marginal extremes independently [14]. Later variants extend this family through suspicion scoring, regularized aggregation, coding-based redundancy, and geometric-median aggregation [15–18, 21]. These methods are attractive because they require little server-side prior knowledge. Under heterogeneous data, however, they can still confuse legitimate benign dispersion with malicious deviation. This limitation has motivated methods that address heterogeneity more directly through optimization stabilization, resampling, bucketing, local-SGD analysis, gradient splitting, nearest-neighbor mixing, label-skew-aware aggregation, and entropy-based public-data guidance [9–11, 22–28].

Trusted-data and history-based defenses. A third line of work augments aggregation with trusted data, history, or auxiliary diagnostics. In collaborative learning, AUROR anticipated update screening for poisoning resistance [29]. In FL, FLTrust introduces a trusted server anchor, while FoolsGold and FLDetector rely on historical similarity diagnostics [19, 30, 31]. Recent work also highlights the cost of robustness itself, including clipping design and the tradeoff between Byzantine resilience and clean accuracy [32, 33]. CARAT is closest to this broader defense-and-detection family, but it uses a different validation principle: instead of reducing trust to a single anchor direction, geometric similarity, or post-hoc clustering signal, it asks whether a client update remains useful across multiple hidden semantic sub-tasks.

3 Preliminaries and Problem Statement

3.1 Federated Learning

We consider synchronous FL with a server and a client population $\mathcal{N} = \{1, \ldots, N\}$. Client i holds a local dataset $\mathcal{D}_i$ and local objective $F_i(w)$. The standard global objective is

$$F(w) = \sum_{i=1}^{N} p_i F_i(w), \qquad p_i = \frac{|\mathcal{D}_i|}{\sum_{j=1}^{N} |\mathcal{D}_j|}. \tag{1}$$

At communication round r, the server broadcasts the current model w_r to selected clients $\mathcal{S}_r$. Each selected client initializes $w_{i,r}^0 = w_r$ and performs K local stochastic-gradient steps,

$$w_{i,r}^{k+1} = w_{i,r}^k - \eta_r \nabla F_i(w_{i,r}^k; \xi_{i,r}^k), \qquad k = 0, \ldots, K-1. \tag{2}$$

The client then returns a model message that the server converts to an update vector

$$\Delta_i^r = w_{i,r}^K - w_r. \tag{3}$$

A server-side aggregation rule maps the received updates to one global step,

$$w_{r+1} = w_r + \mathcal{A}\big(\{\Delta_i^r : i \in \mathcal{S}_r\}\big). \tag{4}$$

FedAvg is recovered when $\mathcal{A}$ is the sample-size-weighted mean. In heterogeneous FL, however, benign updates can differ sharply because client data are non-i.i.d.; this makes geometry-only filtering unreliable as a sole trust signal.

3.2 Threat Model and Server-Side Reference Data

At each round, a subset $\mathcal{M}_r \subseteq \mathcal{S}_r$ of selected clients may be malicious and coordinated. A malicious client may replace its benign update Δ_i^r with an arbitrary crafted update $\widetilde{\Delta}_i^r$ intended to reduce global test accuracy while remaining close enough to the benign update distribution to evade robust aggregation. The server observes only client messages and does not inspect client data.

CARAT assumes that the server maintains a small labeled reference set $\mathcal{D}^{\mathrm{ref}}$ covering the task label space. From this data it constructs a probe pool $\mathcal{P}$ and repeatedly samples hidden, class-balanced validation tasks. This assumption is stronger than fully data-free robust aggregation and should be treated as part of the threat model, not as an incidental implementation detail. The key difference from single-anchor trusted-data methods is that CARAT does not compress $\mathcal{D}^{\mathrm{ref}}$ into one trusted update direction; instead, it uses the reference data to generate multiple hidden semantic checks.

3.3 Problem Statement

The server must assign aggregation weights that suppress harmful updates while preserving legitimate client diversity. Formally, given a round-level update set $\{\Delta_i^r\}_{i \in \mathcal{S}_r}$ and reference data $\mathcal{D}^{\mathrm{ref}}$, the goal is to construct nonnegative weights α_i^r and an aggregate Δ_{agg}^r such that malicious influence is bounded, benign non-i.i.d. specialization is not discarded merely for being geometrically atypical, and the resulting global model maintains clean utility under model-poisoning attacks. CARAT addresses this problem by validating updates through hidden-task utility rather than through centrality alone.

4 CARAT

4.1 Overview

Figure 1 organizes CARAT into four visual blocks: client updates, task-conditioned validation, robust weight optimization, and final aggregation. Algorithmically, these blocks correspond to three coordinated server-side stages. First, the server converts client messages into update vectors, clips their norms robustly, and rescales them to a common evaluation radius. Second, it evaluates each candidate update on multiple hidden class-balanced tasks and combines the resulting utility certificates with a coordinate-rank anomaly penalty. Third, it solves a capped-simplex optimization problem with a temporal prior to produce the aggregation weights and applies them to the clipped updates. The evaluation geometry is normalized across clients, while the final server step remains conservative because it aggregates clipped updates.

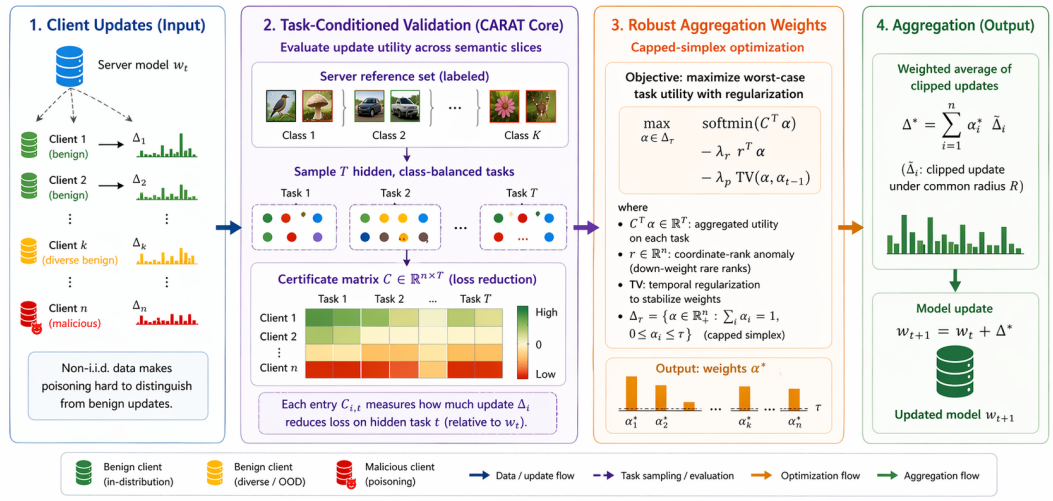

Figure 1: Overview of CARAT. Non-i.i.d. client data can make diverse benign updates difficult to distinguish from poisoning updates. CARAT uses a small labeled server reference set to sample hidden class-balanced tasks, forms a certificate matrix from task-level loss reductions, learns capped-simplex aggregation weights with rank-anomaly and temporal-stability penalties, and updates the global model by a weighted average of clipped client updates.

4.2 Stage 1: Common-Geometry Evaluation

At round r, CARAT first converts client messages into vector-form deltas and computes their norms. Before any semantic evaluation, the server robustly clips client updates so that a small number of large-magnitude messages cannot dominate the comparison stage. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by the MAD-based rule

$$\tau_r = \text{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \text{MAD}(\{\|\Delta_i\|_2\}), \tag{5}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{6}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{7}$$

where q_r is chosen as a quantile of the positive clipped norms. This normalization removes pure scale advantages during hidden-task comparison. The normalized vectors are used only for evaluation; the final model update is still computed from the clipped vectors. CARAT uses normalization to create a fair scoring geometry, not to define the actual optimization step taken by the server.

4.3 Stage 2: Hidden-Task Certificates and Structural Penalties

After preprocessing, CARAT samples T hidden tasks from the probe pool $\mathcal{P}$, with each task constructed to be approximately class-balanced. This is what makes the method class-aware. A malicious update can remain globally plausible while degrading only a subset of classes or semantic regions. Repeated balanced hidden tasks make these localized failures harder to hide. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{8}$$

Here, "certificate" denotes an internal task-level utility score, not a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different intrinsic difficulty and baseline loss scale, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making scores comparable across hidden slices. The resulting matrix gives the server a richer signal than client-to-client similarity alone: a client should be trusted because it remains broadly useful across hidden tasks, not merely because it lies near the geometric center of the client population.

Task utility alone is insufficient, because a malicious client may overfit the probe tasks or align with part of the semantic signal while remaining structurally extreme in parameter space. CARAT therefore augments hidden-task certificates with a coordinate-rank anomaly score. For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, it ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{9}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. A client receives a large anomaly score if its update remains persistently extreme across many coordinates rather than on a few isolated entries. After robust standardization, only positive deviations are retained as penalties. The signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded. Together, the certificate matrix and rank penalty separate two notions that are often conflated in robust aggregation: whether an update is useful and whether it is structurally atypical.

4.4 Stage 3: Capped-Simplex Weight Optimization

CARAT maps these signals to aggregation weights by solving a constrained optimization problem on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{10}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. The cap prevents any single client from dominating the aggregate and encodes a conservative trust budget at the weight level: even when a client appears useful, it cannot absorb arbitrary influence within one round.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. The diagram denotes the temporal penalty by TV; in the implementation and analysis, CARAT uses the quadratic temporal-variation penalty $\mathrm{TV}_2(\alpha, \alpha^{\mathrm{prev}}) = \|\alpha - \alpha^{\mathrm{prev}}\|_2^2$. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{11}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{12}$$

The first term is a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Eq. (11) therefore implements a simple decision rule: a client should retain influence only if it remains useful across hidden tasks and does not repeatedly appear as an outlier in rank space.

After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{13}$$

This is the final server step. The aggregate inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above. In practice, the main additional computation comes from evaluating n candidate updates over T hidden tasks and computing rank

penalties on coordinate subsamples. Probe-model forward passes dominate, while the ranking and optimization stages remain tunable through explicit budget parameters.

A compact round-level algorithm summary is provided in Appendix A. Formal properties of the normalization step, capped-simplex feasibility, softmin behavior, temporal regularization, pairwise weight transfer, malicious-weight influence, and per-round complexity are deferred to Appendix B.

5 Experiments

We evaluate CARAT on class-rich federated benchmarks to test whether hidden-task certificates improve robustness beyond geometry-only and single-anchor defenses while preserving clean utility under heterogeneous client data. The study is organized around three questions: how much clean-utility cost CARAT incurs without attack, how well it resists untargeted model poisoning as heterogeneity increases, and what robustness is obtained per unit of trusted-data budget and runtime overhead. The full benchmark matrix, appendix-only stress tests, and ablation scope are reported in Appendix C.

5.1 Experimental Setup

Benchmarks. The primary benchmark is CIFAR-100 with ResNet18 [34]. CIFAR-100 offers a class-rich label space in which hidden semantic slices are meaningful and class-selective failures are easier to expose than on low-cardinality datasets. We complement it with TinyImageNet, again using ResNet18, to test whether the method scales to a larger class vocabulary while keeping the backbone fixed. To anchor the method in a more application-oriented setting, we also include CHMNIST with the same backbone in the appendix, where class-selective degradation has a clearer domain interpretation. CIFAR-10 is retained only as an appendix sanity check rather than a primary source of evidence, because a ten-class benchmark is too coarse to serve as the main validation vehicle for a class-aware defense. Keeping the model family fixed across visual datasets helps attribute observed differences to aggregation rather than to backbone changes.

Threat models. For untargeted model poisoning, the main threat models are ALIE [7], FangAttack [5], MinMax [6], and MinSum [6]. These attacks represent stealthy optimization-based adversaries that try to stay near the benign update manifold while degrading global learning, together with a geometry-aware attack that is especially informative for defenses based on Krum-style screening. We do not treat data poisoning or backdoor attacks as part of CARAT's claimed scope in this paper.

Baselines. The primary baselines are Mean, NormClipping, TrimmedMean [14], MultiKrum [12], FLTrust [19], FLDetector [31], and CARAT. This set spans plain averaging, norm clipping, coordinate-wise robust statistics, geometry-based filtering, trusted-data anchoring, history-based detection, and class-aware hidden-task validation. We exclude unpublished or under-review defenses from the main comparison so that all baseline claims are tied to citable public methods.

Implementation details. Across the primary comparison, we use 20 clients with full participation, 200 global rounds, 4 local epochs, and learning rate 0.05. To probe heterogeneity, we evaluate Dirichlet non-i.i.d. regimes with $\alpha \in \{1, 0.5\}$ in the main table; the more severe $\alpha = 0.1$ regime is treated as a stress-test setting and is not used for the central claim. The adversarial setting uses an attacker ratio of 20%, corresponding to 4 malicious clients. Clean-reference experiments are reported separately from attacked experiments instead of being inserted as attack rows. Unless otherwise stated, baseline means and standard deviations are computed over 5 seeds. CARAT seed counts are stated in the relevant table captions.

Metrics and fairness. Because CARAT is explicitly class-aware, the evaluation must report more than aggregate clean accuracy. For clean utility, we report test accuracy and test loss under no attack. For robustness, we additionally report macro-average accuracy and worst-class accuracy on class-rich benchmarks so that localized failures remain visible. Fairness requires explicit control of server-side supervision: CARAT and FLTrust are compared under a matched labeled-data budget, and the reference budget is parameterized per class rather than fixed as a dataset-independent total pool size.

Table 1: Experimental setup for evaluating CARAT.

Component	Design choice
Primary benchmark	CIFAR-100 with ResNet18
Supplementary benchmarks	TinyImageNet with ResNet18; CHMNIST with ResNet18 (appendix); CIFAR-10 only as appendix sanity check
Clients and participation	20 total clients with full participation each round
Optimization schedule	200 communication rounds, 4 local epochs, learning rate 0.05
Heterogeneity settings	Dirichlet non-i.i.d. with $\alpha \in \{1, 0.5\}$ for the main comparison; $\alpha = 0.1$ reserved for stress testing
Main adversarial fraction	20% malicious clients
Stress adversarial fraction	40% malicious clients (appendix)
Main untargeted attacks	ALIE, FangAttack, MinMax, MinSum
Untargeted baselines	Mean, NormClipping, TrimmedMean, MultiKrum, FLTrust, FLDetector, CARAT
Primary reported metrics	Final test accuracy, final test loss, average rank, runtime per round where available
Fairness controls	Matched labeled server-reference budget between CARAT and FLTrust; clean-reference runs reported separately from attacked runs
Repetitions	Five seeds for baseline rows; CARAT seed counts reported in the table captions

5.2 Main Evaluation Comparisons

Clean utility. We first report no-attack performance to make the cost of robustness visible. These comparisons focus on test accuracy and test loss, with macro and worst-class accuracy added on class-rich benchmarks when aggregate accuracy alone could mask concentrated degradation.

Untargeted poisoning. We then compare CARAT against Mean, NormClipping, TrimmedMean, MultiKrum, FLTrust, and FLDetector under ALIE, FangAttack, MinMax, and MinSum on non-i.i.d. partitions. The main question is whether task-conditioned validation remains reliable when benign clients are dispersed and adaptive attackers stay close to the benign update manifold.

5.3 Ablations, Stress Tests, and Overhead

Ablations. We ablate the components that correspond directly to CARAT's mechanism: hidden-task count, reference budget per class, rank-penalty weight, temporal-prior weight, clipping controls, and common-radius evaluation. These studies isolate whether the observed robustness comes from class-balanced task validation, structural anomaly suppression, or simply from access to more trusted data.

Stress tests. Appendix-only stress tests raise the attacker fraction to 40%, examine the more severe $\alpha = 0.1$ heterogeneity regime, and vary the client count or class vocabulary. Their role is to probe failure modes and scaling behavior rather than to replace the main-text comparisons.

Overhead analysis. Since Proposition B.9 identifies probe-model evaluation as the main extra cost, we report runtime per round, total defense overhead, and trusted-data budget sensitivity as first-class evaluation outcomes rather than as implementation footnotes.

5.4 Protocol-Locked Evaluation Status

The main empirical comparison is a non-i.i.d. CIFAR-100/ResNet18 evaluation with 20 clients, 20% malicious clients, 200 rounds, and four untargeted model-poisoning attacks. The final NeurIPS table uses only public, citable baselines and matches the labeled reference budget between CARAT and FLTrust. Table 2 is retained as a current result audit: it records reusable legacy baseline results and the earlier single-task CARAT pilot, but it is not the final claim table. The final table will replace the pilot CARAT row with the full $T = 8$ CARAT protocol and will replace the legacy FLTrust row with the matched-budget FLTrust protocol.

Table 2: Current final-accuracy audit (%) under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Higher is better. Published legacy baseline cells use five seeds where available. The CARAT row is the earlier single-task pilot ($T = 1$), not the final $T = 8$ CARAT protocol. The matched-budget FLTrust and final CARAT rows are being regenerated under the locked public-baseline protocol.

Dirichlet α	Defense	ALIE	FangAttack	MinMax	MinSum	Avg. rank
1.0	Mean	49.43 ± 0.39	48.86 ± 0.70	50.49 ± 0.59	50.50 ± 0.49	3.0
1.0	NormClipping	25.27 ± 0.70	22.67 ± 0.77	24.86 ± 0.65	24.74 ± 0.63	7.0
1.0	TrimmedMean		*running under final public-baseline protocol*			
1.0	MultiKrum	39.59 ± 0.74	37.18 ± 0.83	48.69 ± 0.84	47.71 ± 1.12	4.8
1.0	FLTrust (legacy budget)	33.26 ± 0.63	33.81 ± 0.99	33.61 ± 0.86	33.85 ± 0.78	5.8
1.0	FLDetector	47.96 ± 0.58	28.54 ± 4.42	50.58 ± 0.76	50.47 ± 0.27	4.0
1.0	**CARAT pilot** ($T = 1$)	**52.92±1.71**	**53.96±0.86**	**54.20±1.39**	**54.23±1.73**	**1.0**
0.5	Mean	47.62 ± 0.73	47.53 ± 1.01	48.88 ± 0.32	48.88 ± 0.52	3.2
0.5	NormClipping	23.73 ± 0.53	20.64 ± 1.41	23.28 ± 0.40	22.92 ± 0.50	7.0
0.5	TrimmedMean		*running under final public-baseline protocol*			
0.5	MultiKrum	37.00 ± 0.81	33.05 ± 1.20	46.59 ± 1.09	46.45 ± 0.80	4.8
0.5	FLTrust (legacy budget)	31.72 ± 0.51	32.20 ± 0.47	32.21 ± 0.68	32.08 ± 0.46	5.8
0.5	FLDetector	45.30 ± 0.75	27.77 ± 1.25	48.85 ± 0.53	49.01 ± 0.69	4.2
0.5	**CARAT pilot** ($T = 1$)	**51.89±0.56**	**52.58±1.30**	**53.76±0.36**	**54.81**	**1.0**

The audit table is useful for checking that the benchmark pipeline and legacy baseline parser agree with the intended CIFAR-100 protocol, but we do not use it as the final NeurIPS evidence because two cells violate the locked protocol: the CARAT row uses a single hidden task rather than the full $T = 8$ setting, and the FLTrust row uses a smaller trusted-data budget than CARAT. The final public-baseline run therefore regenerates CARAT, FLTrust, and TrimmedMean under the same protocol while reusing only compatible published-baseline results.

Figure 2 gives the corresponding round-wise view for the single-task FangAttack pilot. The final figure will be regenerated from the full $T = 8$ CARAT runs once the locked public-baseline experiment queue completes.

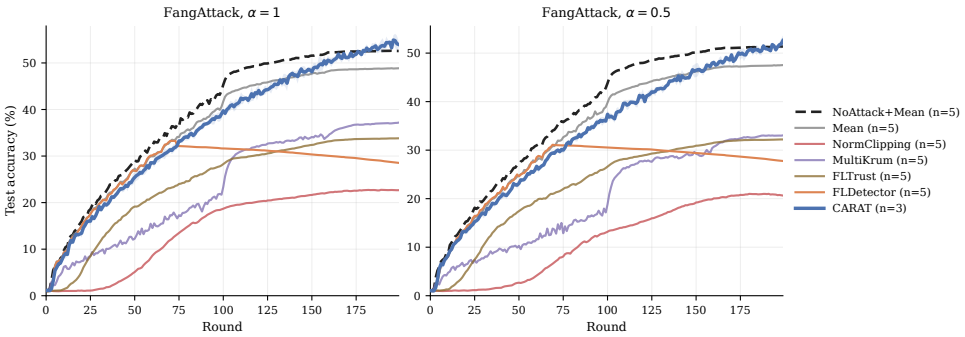

Figure 2: Evolution of test accuracy on CIFAR-100/ResNet18 under FangAttack with 20% malicious clients and non-i.i.d. data. Curves show the mean over reported seeds; the CARAT band shows one standard deviation. The dashed black curve is the no-attack Mean reference under the same client partition protocol.

The corresponding final-loss audit is reported in Appendix C. As with the accuracy audit, those loss values are retained only to document the legacy pilot state; the final loss table will be regenerated from the locked $T = 8$ CARAT and matched-budget FLTrust runs. Additional verification-scale learning curves and runtime measurements are reported in Appendix C.8.

6 Limitations

CARAT is not a data-free defense. It assumes a labeled server-side reference set that covers the task label space, so it is most appropriate for cross-silo or institutionally managed FL settings where audit

data are available. If the reference set is biased, too small, or missing important classes, hidden-task certificates may overestimate the utility of harmful updates outside the reference support.

The main empirical claim is also deliberately scoped. The matched comparison evaluates untargeted model poisoning on CIFAR-100/ResNet18 under $\alpha \in \{1, 0.5\}$ and a 20% attacker fraction. The experiments do not establish universal robustness against backdoor attacks, data poisoning, or all possible adaptive attacks against the hidden-task mechanism. More severe heterogeneity, larger client populations, and additional datasets should be treated as stress tests rather than as claims already settled by the main table.

Finally, CARAT adds server-side computation because every round evaluates candidate updates on multiple hidden tasks. The overhead is tunable through the number of tasks, probe samples, coordinate subsamples, and optimization steps, but deployments with strict server latency constraints may require caching, batching, or a smaller probe schedule.

7 Conclusion

We presented CARAT, a class-aware robust aggregation rule that validates client updates through hidden task certificates rather than relying only on geometric centrality or a single trusted reference direction. By scoring updates across multiple hidden, class-balanced semantic slices and combining those scores with rank-space anomaly suppression and capped-simplex weight optimization, CARAT is designed to preserve benign heterogeneity while reducing the influence of semantically harmful or structurally extreme clients.

The CIFAR-100/ResNet18 non-i.i.d. evaluation is scoped to public robust-aggregation baselines, matched trusted-data budgets, and untargeted poisoning attacks. The evidence should therefore be interpreted as a heterogeneous untargeted-poisoning result rather than as a universal poisoning claim. The method is most appropriate in FL deployments where the server can maintain a modest labeled audit set and where class-selective failures matter under heterogeneous client drift. The next empirical step after the locked public-baseline runs is to broaden the stress-test suite across more severe heterogeneity, larger client populations, trusted-budget sweeps, and additional attack families.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. *Advances and Open Problems in Federated Learning*. Now Publishers, 2021.

[4] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643, 2019.

[5] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[6] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[8] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, et al. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[9] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, 2020.

[10] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*, pages 5132–5143, 2020.

[11] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. Federated learning on non-IID data: A survey. *Neurocomputing*, 465:371–390, 2021.

[12] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[13] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[14] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[15] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *International Conference on Machine Learning*, pages 6893–6901, 2019.

[16] Lingjiao Chen, Hongyi Wang, Zachary Charles, and Dimitris Papailiopoulos. DRACO: Byzantine-resilient distributed training via redundant gradients. In *International Conference on Machine Learning*, pages 903–912, 2018.

[17] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *AAAI Conference on Artificial Intelligence*, pages 1544–1551, 2019.

[18] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70:1142–1154, 2022.

[19] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[20] Pengrui Liu, Xiangrui Xu, and Wei Wang. Threats, attacks and defenses to federated learning: Issues, taxonomy and perspectives. *Cybersecurity*, 5(4), 2022.

[21] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Defending against saddle point attack in byzantine-robust distributed learning. In *International Conference on Machine Learning*, pages 7074–7084, 2019.

[22] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via resampling. In *International Conference on Learning Representations*, 2021.

[23] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[24] Deepesh Data and Suhas Diggavi. Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data. In *International Conference on Machine Learning*, pages 2478–2488, 2021.

[25] Yuchen Liu, Chen Chen, Lingjuan Lyu, Fangzhao Wu, Sai Wu, and Gang Chen. Byzantine-robust learning on heterogeneous data via gradient splitting. In *International Conference on Machine Learning*, pages 21404–21425, 2023.

[26] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafael Pinot, and John Stephan. Fixing by mixing: A recipe for optimal byzantine ML under heterogeneity. In *International Conference on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.

[27] Wenxuan Bao, Jun Wu, and Jingrui He. BOBA: Byzantine-robust federated learning with label skewness. In *International Conference on Artificial Intelligence and Statistics*, pages 892–900, 2024.

[28] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *International Conference on Machine Learning*, pages 20096–20110, 2024.

[29] Shiqi Shen, Shruti Tople, and Prateek Saxena. AUROR: Defending against poisoning attacks in collaborative deep learning systems. In *Annual Computer Security Applications Conference*, 2016.

[30] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[31] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[32] Youssef Allouah, Rachid Guerraoui, Nirupam Gupta, Ahmed Jellouli, Geovani Rizk, and John Stephan. Adaptive gradient clipping for robust federated learning. In *International Conference on Learning Representations*, 2025.

[33] Yi-Rui Yang, Chang-Wei Shi, and Wu-Jun Li. On the tension between byzantine robustness and no-attack accuracy in distributed learning. In *International Conference on Machine Learning*, 2025.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Algorithm 1 CARAT server aggregation at round r

Require: Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, probe pool $\mathcal{P}$, CARAT hyperparameters
Ensure: Aggregated update Δ_{agg}, updated temporal prior α^{prev}

```
if P is uninitialized then
    Build a balanced probe pool from the server-visible training dataset
end if
if r = 1 then
    α^prev ← (1/n) 1
end if
for i = 1 to n do
    Convert u_i^r into a vector-form delta Δ_i^r
end for
Compute the clipping threshold τ_r from {‖Δ_i^r‖_2}_{i=1}^n
for i = 1 to n do
    Δ̄_i^r ← min(1, τ_r / (‖Δ_i^r‖_2 + ε)) Δ_i^r
end for
Choose the evaluation radius q_r from a quantile of positive clipped norms
for i = 1 to n do
    Δ̃_i^r ← q_r Δ̄_i^r / (‖Δ̄_i^r‖_2 + ε)
end for
Sample T class-balanced hidden tasks from P with replacement/subset fallback if strict balanced
sampling fails
for t = 1 to T do
    Evaluate the baseline loss ℓ_t(w_r)
end for
for i = 1 to n do
    for t = 1 to T do
        c_{i,t} ← ℓ_t(w_r) − ℓ_t(w_r + Δ̃_i^r)
    end for
end for
Robustly standardize certificate columns to obtain C^std
Compute rank penalties r from repeated coordinate subsamples of {Δ̄_i^r}_{i=1}^n
α^(0) ← α^prev
for k = 0 to K − 1 do
    z ← (C^std)^⊤ α^(k)
    π ← softmin task weights induced by z and β
    g ← C^std π − λ_r r − 2λ_p (α^(k) − α^prev)
    α^(k+1) ← Π_{A(ρ̂)}(α^(k) + ηg)
end for
α* ← α^(K) and α^prev ← α*
Δ_agg ← Σ_{i=1}^n α_i* Δ̄_i^r
return Δ_agg, α^prev
```

A Algorithmic Details

A.1 Round-Level Pseudocode

Algorithm 1 summarizes one CARAT server round in standard pseudocode form.

A.2 Implementation Notes

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior for the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This keeps the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Reference-pool construction and hidden-task sampling. Let $\mathcal{P}_c \subseteq \mathcal{P}$ denote the subset of probe examples with class label c. A hidden task is constructed by sampling a small number of examples from each available class pool and concatenating them into one balanced micro-task. If the desired number of samples per class is not available without replacement, CARAT falls back to replacement and, if needed, to a random subset of the probe pool rather than aborting the round. This fail-soft design preserves the intended semantic diversity of the task sampler while ensuring that the server always obtains a well-defined certificate matrix.

Robust score calibration. For notational economy, the main text reuses C after standardization. In implementation terms, the raw certificate matrix is first transformed column-wise into

$$C^{\mathrm{std}}_{i,t} = \frac{c_{i,t} - \mathrm{median}_j(c_{j,t})}{1.4826 \cdot \mathrm{MAD}_j(c_{j,t}) + \varepsilon}.$$

This removes task-specific scale effects while preserving within-task ordering by Proposition B.4. Rank penalties are calibrated similarly. For each coordinate subsample S_s, let

$$\rho_i^{(s)} = \frac{1}{|S_s|} \sum_{j \in S_s} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|.$$

CARAT then computes a standardized rank score

$$z_i^{(s)} = \frac{\rho_i^{(s)} - \mathrm{median}_\ell(\rho_\ell^{(s)})}{1.4826 \cdot \mathrm{MAD}_\ell(\rho_\ell^{(s)}) + \varepsilon},$$

and aggregates subsamples through

$$r_i = \max\left(0, \mathrm{median}_s\, z_i^{(s)}\right).$$

As a result, a client is penalized only when it is consistently extreme across repeated coordinate views, not because of a single noisy subsample.

Projected optimization details. Again writing C for the standardized certificate matrix, CARAT initializes the optimizer with $\alpha^{(0)} = \alpha^{\mathrm{prev}}$ after the first round, and with the uniform vector at initialization. If

$$z(\alpha) = C^\top \alpha, \qquad \pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then the ascent direction used in projected updates is

$$g(\alpha) = C\pi(\alpha) - \lambda_r r - 2\lambda_p(\alpha - \alpha^{\mathrm{prev}}).$$

With optimizer step size η, one iteration takes the form

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta g(\alpha^{(k)})\right).$$

The projection maintains feasibility throughout optimization, so every intermediate iterate already satisfies the nonnegativity, unit-sum, and per-client cap constraints described in the main text.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

Practical diagnostics. The most useful round-level diagnostics for debugging and empirical interpretation are the learned weights, the weakest-task values emphasized by the softmin, the clip threshold τ_r, the evaluation radius q_r, and the final rank penalties r_i. In successful runs, these quantities help distinguish three qualitatively different cases: benign but diverse clients that retain moderate weight, clearly harmful clients that lose certificate support across multiple tasks, and structurally extreme clients whose influence is reduced primarily by the rank penalty.

B Theoretical Analysis

This appendix records the main structural properties used to interpret CARAT. These results are not presented as a complete robustness guarantee against arbitrary adaptive attackers. Instead, they clarify why the method decomposes into a normalized evaluation geometry, a task-aware utility objective, and a constrained robust aggregation step.

Proposition B.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition B.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \min_t s_t + \frac{\log T}{\beta}.$$

Moreover,

$$\lim_{\beta \to \infty} \operatorname*{softmin}_{\beta}(s) = \min_t s_t, \qquad \lim_{\beta \to 0} \operatorname*{softmin}_{\beta}(s) = \frac{1}{T} \sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. Let $m = \min_t s_t$. Since

$$e^{-\beta m}/T \leq \frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \leq e^{-\beta m},$$

applying $-(1/\beta)\log(\cdot)$ yields

$$m \le \operatorname*{softmin}_{\beta}(s) \le m + \frac{\log T}{\beta}.$$

The limit $\beta \to \infty$ follows immediately from the vanishing gap bound. For $\beta \to 0$, a first-order expansion of the exponential average gives

$$\frac{1}{T}\sum_{t=1}^{T} e^{-\beta s_t} = 1 - \beta \frac{1}{T}\sum_{t=1}^{T} s_t + o(\beta),$$

so the softmin converges to the arithmetic mean. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition B.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (7) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \dots, T\}.$$

Proof sketch. Eq. (7) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (8) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition B.4 (Task-wise standardization preserves within-task ordering)**.** *Fix a task t and define a standardized certificate score*

$$\hat{c}_{i,t} = \frac{c_{i,t} - m_t}{s_t + \varepsilon},$$

where $m_t \in \mathbb{R}$ and $s_t > 0$ depend only on task t and not on client i. Then for any clients i, j,

$$c_{i,t} \ge c_{j,t} \quad \Longleftrightarrow \quad \hat{c}_{i,t} \ge \hat{c}_{j,t}.$$

Moreover,

$$\hat{c}_{i,t} - \hat{c}_{j,t} = \frac{c_{i,t} - c_{j,t}}{s_t + \varepsilon}.$$

Proof sketch. For a fixed task t, the map $x \mapsto (x - m_t)/(s_t + \varepsilon)$ is affine with strictly positive slope because $s_t + \varepsilon > 0$. Hence it preserves ordering. The difference identity follows by direct subtraction. Therefore column-wise robust standardization calibrates cross-task scale without changing the within-task ranking induced by the raw certificate scores. □

Proposition B.5 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition B.6 (Pairwise weight-transfer condition)**.** *Let*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2$$

be the CARAT objective, and let e_a, e_b be coordinate vectors for two clients a and b. Suppose $\alpha + \epsilon(e_b - e_a)$ remains feasible for sufficiently small $\epsilon > 0$. Define

$$\pi_t(\alpha) = \frac{\exp(-\beta(C^\top \alpha)_t)}{\sum_{\ell=1}^{T} \exp(-\beta(C^\top \alpha)_\ell)}.$$

Then moving infinitesimal weight from client a to client b increases the objective to first order whenever

$$(C_b - C_a)^\top \pi(\alpha) - \lambda_r(r_b - r_a) - 2\lambda_p[(\alpha_b - \alpha_b^{\text{prev}}) - (\alpha_a - \alpha_a^{\text{prev}})] > 0,$$

where C_i denotes row i of the standardized certificate matrix.

Proof sketch. Let $d = e_b - e_a$. A first-order expansion gives

$$f(\alpha + \epsilon d) - f(\alpha) = \epsilon d^\top \nabla f(\alpha) + o(\epsilon).$$

Using Proposition B.2, the gradient is

$$\nabla f(\alpha) = C\pi(\alpha) - \lambda_r r - 2\lambda_p(\alpha - \alpha^{\text{prev}}).$$

Substituting $d = e_b - e_a$ yields exactly the displayed expression. Hence a client gains weight when its hidden-task utility advantage, measured through the current softmin focus $\pi(\alpha)$, outweighs its rank penalty and temporal-prior cost. □

Proposition B.7 (Norm bound for the final aggregate)**.** *Suppose each clipped update satisfies $\|\bar{\Delta}_i\|_2 \leq \tau_r$ and $\alpha \in \mathcal{A}(\hat{\rho})$. Then the final aggregate*

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i$$

satisfies

$$\|\Delta_{\text{agg}}\|_2 \leq \tau_r.$$

Proof sketch. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the aggregate is a convex combination of clipped updates. By the triangle inequality,

$$\|\Delta_{\text{agg}}\|_2 \leq \sum_{i=1}^{n} \alpha_i \|\bar{\Delta}_i\|_2 \leq \sum_{i=1}^{n} \alpha_i \tau_r = \tau_r.$$

Thus the final server step inherits the clipping radius as a hard norm upper bound. □

Proposition B.8 (Influence bound from malicious weight mass)**.** *Let $\mathcal{M}$ be the malicious clients selected in a round and let*

$$q_{\mathcal{M}} = \sum_{i \in \mathcal{M}} \alpha_i$$

be their total CARAT weight. Suppose every clipped update satisfies $\|\bar{\Delta}_i\|_2 \leq \tau_r$. Then the direct malicious contribution obeys

$$\left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2 \leq q_{\mathcal{M}} \tau_r.$$

If $q_{\mathcal{M}} < 1$ and

$$\Delta_{\mathcal{B}} = \sum_{i \notin \mathcal{M}} \frac{\alpha_i}{1 - q_{\mathcal{M}}} \bar{\Delta}_i$$

denotes the benign-only aggregate obtained by renormalizing the benign weights, then

$$\|\Delta_{\text{agg}} - \Delta_{\mathcal{B}}\|_2 \leq 2 q_{\mathcal{M}} \tau_r.$$

Proof sketch. The first inequality follows from the triangle inequality:

$$\left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2 \leq \sum_{i \in \mathcal{M}} \alpha_i \|\bar{\Delta}_i\|_2 \leq q_{\mathcal{M}} \tau_r.$$

For the second inequality, write

$$\Delta_{\text{agg}} = (1 - q_{\mathcal{M}})\Delta_{\mathcal{B}} + \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i.$$

Therefore

$$\|\Delta_{\text{agg}} - \Delta_{\mathcal{B}}\|_2 \leq q_{\mathcal{M}} \|\Delta_{\mathcal{B}}\|_2 + \left\| \sum_{i \in \mathcal{M}} \alpha_i \bar{\Delta}_i \right\|_2 .$$

Since $\Delta_{\mathcal{B}}$ is a convex combination of clipped benign updates, $\|\Delta_{\mathcal{B}}\|_2 \leq \tau_r$ by the same argument as Proposition B.7. Combining this with the first inequality gives the bound. □

Proposition B.9 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the prototype implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The object $\mathcal{D}^{\text{ref}}$ in the main text denotes the labeled server reference set, while the probe pool $\mathcal{P}$ used during each round is a subsampled working set derived from it. In practice, the server first constructs $\mathcal{P}$ from the full server-visible reference data and then samples hidden tasks from $\mathcal{P}$ at each round. This distinction should remain explicit because it is part of the method's server-side assumption and materially strengthens the setting relative to a fully data-free defense.

C Additional Experimental Details

This appendix records the benchmark design, sensitivity analyses, and reporting scope that support the main-text experimental design.

C.1 Experimental Goals

The empirical study is designed to test properties specific to CARAT rather than to benchmark it as a generic outlier filter. In particular, we ask whether hidden-task validation improves robustness when benign clients are heterogeneous, whether that advantage persists on class-rich benchmarks where class-selective failure matters, and how much robustness is purchased per unit of trusted-data budget and runtime overhead.

C.2 Benchmarks

Primary class-rich benchmark. The main benchmark is CIFAR-100 with ResNet18. This setting best matches CARAT's motivation because the label space is rich enough for different hidden tasks to emphasize different semantic slices, and because average clean accuracy alone can hide substantial per-class degradation.

Scalability benchmark. TinyImageNet with ResNet18 is used to test whether the same aggregation rule remains effective as the number of classes and the visual vocabulary increase. This benchmark is particularly useful for checking whether the hidden-task construction and reference-budget design remain meaningful beyond CIFAR-100.

Application-oriented benchmark. We include CHMNIST with ResNet18 as an application-style benchmark in the appendix. Its role is not to maximize scale, but to show that class-selective failures have a practical interpretation in a domain where server-side auditing with a small labeled reference set is plausible.

Sanity-check benchmark. CIFAR-10 is retained only as a sanity-check benchmark. It remains useful for verifying that CARAT does not fail on standard low-cardinality image classification, but it is not treated as a primary source of evidence because its ten classes are too few to fully exercise the method's class-aware design.

C.3 Threat Models

Untargeted poisoning. The main untargeted suite contains ALIE, FangAttack, MinMax, and MinSum. Together they represent adaptive model-poisoning attacks that aim to remain stealthy under server-side defenses, plus a geometry-aware stressor that is especially informative for Krum-style and history-based defenses.

C.4 Baselines

Untargeted defenses. For untargeted poisoning, the primary comparison uses Mean, NormClipping, TrimmedMean, MultiKrum, FLTrust, FLDetector, and CARAT. This set spans plain averaging, norm-based clipping, coordinate-wise robust statistics, geometry-based filtering, trusted-data anchoring, historical-trajectory detection, and class-aware hidden-task validation. Under-review defenses are excluded from the paper's main claim set.

C.5 Metrics and Fairness

No-attack clean utility is reported separately from attacked runs. The clean metrics are test accuracy and test loss. On class-rich datasets we additionally report macro-average accuracy and worst-class accuracy, because the motivation for CARAT is precisely that averaging can hide semantically concentrated failures. Runtime per round and total defense overhead are reported for all defenses that add nontrivial server-side computation.

Trusted-data fairness is controlled explicitly. CARAT and FLTrust are compared under the same labeled server-reference budget, rather than allowing one method to benefit from more public data. In addition to fixed-budget comparisons, we sweep the reference budget in the appendix to show whether CARAT's advantage comes from the hidden-task validation principle rather than from budget alone.

C.6 Shared Training Protocol

Unless stated otherwise, the primary comparison uses 20 clients with full participation, 200 communication rounds, 4 local epochs, and learning rate 0.05. The main heterogeneity settings are Dirichlet non-i.i.d. partitions with $\alpha \in \{1, 0.5\}$; the more severe $\alpha = 0.1$ setting is reserved for stress testing rather than used as a main-text claim. The main attacker ratio is 20%, while 40% is reserved for appendix stress tests. Baseline rows use five random seeds. CARAT seed counts are reported in the relevant table captions. Scaling studies in the appendix may additionally vary the number of clients or the attacker ratio, but these are treated as stress tests rather than as the core evaluation axis.

C.7 Final Loss Results

Table 3 reports the current final-loss audit corresponding to Table 2. As with the accuracy table, it documents the legacy pilot state and will be regenerated from the locked $T = 8$ CARAT and matched-budget FLTrust runs before the final submission.

C.8 Verification-Scale Runtime Results

Figure 3 reports a smaller CIFAR-100/i.i.d. ALIE verification run that records both round-wise accuracy and server-side runtime. In this setting, CARAT remains stable across seeds under attack

Table 3: Current final-loss audit under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Lower is better. Published legacy baseline cells use five seeds where available. The CARAT row is the earlier single-task pilot ($T = 1$), not the final $T = 8$ CARAT protocol.

Dirichlet α	Defense	ALIE	FangAttack	MinMax	MinSum	Avg. rank
1.0	Mean	0.0292 ± 0.0004	0.0294 ± 0.0004	0.0283 ± 0.0004	0.0283 ± 0.0006	3.2
1.0	NormClipping	0.0476 ± 0.0005	0.0501 ± 0.0004	0.0484 ± 0.0004	0.0485 ± 0.0003	7.0
1.0	TrimmedMean	*running under final public-baseline protocol*				
1.0	MultiKrum	0.0361 ± 0.0006	0.0377 ± 0.0005	0.0295 ± 0.0005	0.0301 ± 0.0009	4.8
1.0	FLTrust (legacy budget)	0.0409 ± 0.0004	0.0407 ± 0.0006	0.0406 ± 0.0007	0.0406 ± 0.0005	5.8
1.0	FLDetector	0.0305 ± 0.0004	0.0450 ± 0.0035	0.0283 ± 0.0006	0.0284 ± 0.0003	4.0
1.0	**CARAT pilot** ($T = 1$)	**0.0267±0.0009**	**0.0260±0.0005**	**0.0260±0.0007**	**0.0260±0.0010**	**1.0**
0.5	Mean	0.0303 ± 0.0005	0.0302 ± 0.0006	0.0293 ± 0.0001	0.0293 ± 0.0004	3.2
0.5	NormClipping	0.0486 ± 0.0005	0.0517 ± 0.0011	0.0497 ± 0.0005	0.0498 ± 0.0004	7.0
0.5	TrimmedMean	*running under final public-baseline protocol*				
0.5	MultiKrum	0.0382 ± 0.0008	0.0407 ± 0.0010	0.0306 ± 0.0006	0.0308 ± 0.0006	4.8
0.5	FLTrust (legacy budget)	0.0420 ± 0.0005	0.0417 ± 0.0005	0.0417 ± 0.0006	0.0417 ± 0.0004	5.8
0.5	FLDetector	0.0319 ± 0.0004	0.0458 ± 0.0011	0.0294 ± 0.0004	0.0292 ± 0.0005	4.0
0.5	**CARAT pilot** ($T = 1$)	**0.0271±0.0006**	**0.0266±0.0010**	**0.0259±0.0003**	**0.0253**	**1.0**

but pays a clear runtime premium because each round evaluates candidate updates on repeated hidden tasks. The result is not used as the main robustness claim; it is included to make the overhead tradeoff visible and to motivate the budget controls in the ablation plan.

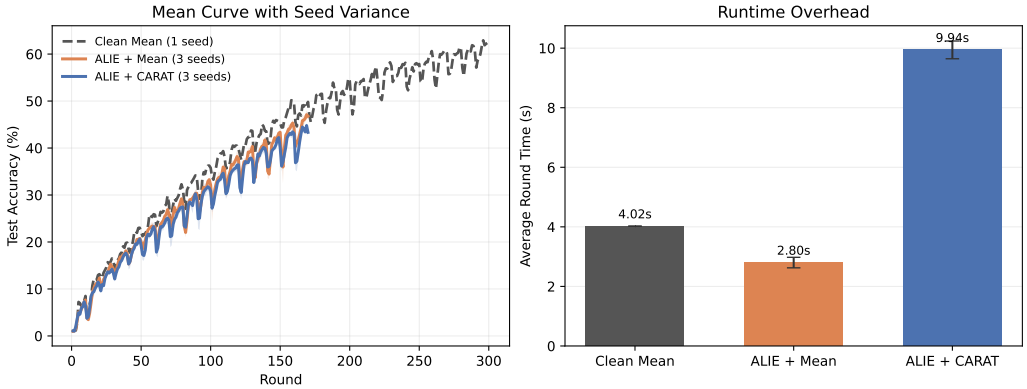

Figure 3: Verification-scale local results on CIFAR-100 under the ALIE model-poisoning attack. Left: round-wise test accuracy, where solid curves show the mean over three seeds and shaded bands denote one standard deviation; the dashed clean Mean curve is a single-seed no-attack reference. Right: average per-round wall-clock time with seed-level error bars, showing the defense overhead of repeated hidden-task evaluation.

C.9 Supplementary Experiment Scope

The main-text comparison is intentionally narrower than the full experimental implementation. In particular, the implementation supports a broader set of model-poisoning attacks and defense baselines than the primary CARAT comparison uses. Table 4 separates the core suite used for the main claim from additional experiment axes that are most useful for stress testing, sensitivity analysis, and future extensions. Backdoor and data-poisoning implementations remain outside the claim set of this paper.

C.10 CARAT Hyperparameters, Stress Tests, and Ablation Plan

The implementation defaults are 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 projected optimization steps, and optimizer learning rate 0.5.

Table 4: Supplementary model-poisoning experiment scope. The core suite matches the main-text claim, while the extension suite records additional comparisons supported by the implementation.

Axis	Core suite	Extension suite	Suggested visualization
Untargeted attacks	ALIE, FangAttack, MinMax, MinSum	Gaussian, IPM, Mimic, SignFlipping, ThreeDFed	Final robustness tables; round-wise accuracy curves
Untargeted defenses	Mean, NormClipping, TrimmedMean, MultiKrum, FLTrust, FLDetector, CARAT	Median, Krum, Bulyan, RFA, DnC, Auror, LASA, FoolsGold, SignGuard, CenteredClipping, Bucketing, SimpleClustering	Grouped comparison tables; per-attack win/loss summaries
Data regimes	Dirichlet non-i.i.d. $\alpha \in \{1, 0.5\}$	$\alpha = 0.1$, i.i.d. verification runs, high-attacker stress tests (40%), client-count scaling, CIFAR-10 sanity checks, TinyImageNet, CHMNIST	Robustness-vs-heterogeneity curves; benchmark summary tables
Efficiency and fairness	Matched CARAT / FLTrust reference budget; runtime per round	Reference-budget sweeps; runtime breakdowns; trusted-data sensitivity	Overhead bars; budget-response curves; clean-robustness-overhead tradeoff plots
Out-of-scope implemented attacks and defenses	–	ModelReplacement, DBA, Neurotoxin, BadNets, EdgeCase, AlterMin, LabelFlipping, FLAME, DeepSight, CRFL	Keep outside the claim set unless the paper scope is widened

Two design principles matter when transferring these defaults across datasets. First, the reference budget should scale with the number of classes rather than be fixed as a single dataset-independent total pool size. Second, CARAT should be ablated along the axes that correspond directly to its mechanism: hidden-task count, reference budget per class, rank-penalty weight, temporal-prior weight, and clipping controls. The appendix therefore includes ablations that isolate whether the observed robustness comes from class-balanced hidden tasks, from the structural rank penalty, or simply from access to more trusted data. It also records high-attacker stress tests, scaling analyses, and runtime breakdowns so that robustness, efficiency, and trusted-data dependence are evaluated together.

D Reproducibility, Ethics, and Availability

Reproducibility. All main reported results use public benchmark datasets and fixed training protocols described in Section 5. The aggregation method is specified in Algorithm 1, with implementation defaults listed above. For anonymous review, logs, plotting scripts, and configuration files should be included in the supplementary material or an anonymized archive so that each table cell can be traced to its seed-level runs.

Compute resources. Experiments were launched on a Compute Canada SLURM cluster through the repository launch scripts. The primary CIFAR-100/CARAT 200-round runs requested one NVIDIA H100 80GB HBM3 MIG GPU slice with the SLURM resource string `nvidia_h100_80gb_hbm3_2g.20gb:1`, 8 CPU cores, 64GB host memory, and a 24 hour wall-time limit per array task; the launcher used 4 data-loading workers and capped concurrent array tasks at 3 for this main CARAT specification. Supplementary CARAT specifications used the same single-GPU resource profile with wall-time limits between 4 hours and 2 days and array concurrency between 1 and 4, depending on the benchmark and sweep size. Each run writes a metadata file recording the configuration file, command line, seed, Python environment, CUDA requirement, and git commit.

Ethics. This work studies defenses against model-poisoning attacks in federated learning. The attack implementations are used to evaluate robustness and do not require access to private user data. The main practical risk is dual use: stronger attack simulations may help adversaries tune poisoning strategies. We mitigate this by framing attacks as benchmark stressors and by releasing defensive evaluation code together with the corresponding protocols.

Data availability. CIFAR-10, CIFAR-100, TinyImageNet, and CHMNIST are public datasets. CARAT uses a small labeled server-side reference split during evaluation; when reproducing the results, this split should be generated deterministically from the public training data under the same seed and excluded from client local training data.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction state the method, assumptions, benchmark scope, and main result. The limitations section narrows the claim to untargeted model poisoning with server-side reference data.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: Section 6 discusses the server-side reference-data assumption, empirical scope, attack-family scope, and runtime overhead.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes].

Justification: The formal results are stated as propositions with assumptions and proof sketches in Appendix B. They are per-round aggregation properties rather than a full convergence theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 5 and Appendix C specify datasets, attacks, defenses, client counts, training schedule, hyperparameters, seed counts, and reporting protocol for the main results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No].

Justification: The paper describes the required code, configurations, logs, and public datasets, but the final anonymized code archive must still be attached or linked for the submission.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The main text gives the training setup and Table 1 summarizes the protocol. Appendix C adds hyperparameters, stress-test axes, and ablation details.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: Tables report mean and standard deviation over seeds where available, and the captions state CARAT seed counts explicitly. The one single-seed CARAT cell is identified in the main text and is not used for variance-sensitive conclusions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes].

   Justification: Appendix D reports the Compute Canada SLURM setup, GPU type, CPU and memory request, wall-time limits, array concurrency, and run metadata recorded by the launcher.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes].

   Justification: The work uses public datasets and evaluates defenses against simulated attacks. Appendix D discusses ethics and dual-use considerations.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes].

    Justification: Appendix D discusses positive impacts for FL security and the dual-use risk of attack simulations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a high-risk model, scraped dataset, or generative system requiring special access safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The paper cites the benchmark datasets and related methods, but dataset license details and third-party code license details should be verified before final submission.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A].

    Justification: The paper introduces an algorithm and experimental implementation, but it does not introduce a new dataset, pretrained model, or benchmark asset.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A].

    Justification: The work does not involve crowdsourcing or human-subject experiments.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A].

    Justification: The work does not involve crowdsourcing or human-subject experiments, so IRB approval is not applicable.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A].

Justification: LLMs are not part of the core method, experiments, or scientific contribution. Any language editing assistance does not affect the methodology or reported results.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.